Модуль 6

MongoDB и Mongoose

MongoDB • Mongoose • Схемы • Модели • CRUD • Валидация • Связи • Индексы • Агрегация

6.1 Что такое MongoDB

MongoDB — документо-ориентированная NoSQL база данных. В отличие от реляционных баз (PostgreSQL, MySQL), где данные хранятся в таблицах со строгой структурой, MongoDB хранит данные в виде JSON-подобных документов (BSON) в коллекциях.

Преимущества MongoDB:

- Гибкая схема — документы в одной коллекции могут иметь разные поля
- Хранение вложенных объектов и массивов — не нужны JOIN-запросы
- Горизонтальное масштабирование (шардинг)
- Отлично подходит для прототипирования и проектов с изменяющейся структурой данных

6.2 SQL vs NoSQL: в чём разница?

В SQL-базах (PostgreSQL, MySQL и других) данные хранятся в таблицах с жёсткой структурой: каждая строка имеет одинаковые столбцы. Связи между таблицами реализуются через JOIN-запросы. Перед добавлением данных нужно создать схему таблицы (CREATE TABLE).

В MongoDB (NoSQL) данные хранятся в виде документов — объектов, похожих на JSON. Каждый документ может иметь свою структуру — это называется «гибкая схема». Вместо таблиц используются коллекции, а вместо строк — документы.

Сравнение терминов:

SQL	MongoDB
База данных	База данных
Таблица	Коллекция (Collection)
Строка	Документ (Document)
Столбец	Поле (Field)

PRIMARY KEY	id (генерируется автоматически)
JOIN	populate / $lookup

MongoDB хорошо подходит, когда:

- Структура данных может меняться или не все документы имеют одинаковые поля.
- Нужна высокая скорость записи и чтения (блоги, логи, каталоги товаров).
- Данные естественно вложены (комментарии внутри поста, адрес внутри пользователя).
- Вы строите API или микросервис, где данные отдаются как JSON.

SQL-базы лучше подходят, когда:

- Много сложных связей между таблицами (e-commerce с заказами, платежами, складом).
- Критически важна целостность данных (банковские транзакции).
- Нужны сложные аналитические запросы с GROUP BY, HAVING и подзапросами.

6.3 Установка и запуск MongoDB

Существует несколько вариантов установки и запуска MongoDB. Рассмотрим все варианты

Вариант 1. MongoDB Atlas (облачная версия — рекомендуется для обучения):

Зайдите на https://www.mongodb.com/atlas и зарегистрируйтесь.

Создайте бесплатный кластер (Free Tier, M0).

В разделе Security → Database Access создайте пользователя с паролем.

В разделе Security → Network Access добавьте свой IP (или 0.0.0.0/0 для доступа отовсюду).

Нажмите Connect → Drivers → скопируйте строку подключения:

```
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/<dbname>
```

Замените <username>, <password> и <dbname> на свои значения.

Вариант 2. Локальная установка:

Установка локально выполняется разными способами в зависимости от операционной системы.

Установка на macOS

Установите Homebrew (если ещё не установлен): https://brew.sh

Добавьте репозиторий MongoDB и установите:

```
brew tap mongodb/brew
brew install mongodb-community
```

Запустите MongoDB как фоновый сервис:

```
brew services start mongodb-community
```

Проверьте:

```
mongosh
```

Если видите приглашение test> — MongoDB работает.

Для остановки:

```
brew services stop mongodb-community
```

Установка на Windows

Скачайте установщик с официального сайта:
https://www.mongodb.com/try/download/community

Выберите версию для Windows, формат MSI.

Запустите установщик. В процессе установки:

- Выберите «Complete» (полная установка).

- Отметьте галочку «Install MongoDB as a Service» — MongoDB будет запускаться автоматически при старте Windows.

- Отметьте «Install MongoDB Compass» — это графический интерфейс для работы с базой (по желанию).

После установки MongoDB автоматически запустится как служба Windows.

Откройте PowerShell или CMD и проверьте:

```
mongosh
```

Если видите приглашение test> — MongoDB работает.

Если команда mongosh не найдена, добавьте путь к MongoDB в переменную PATH:

```
C:\Program Files\MongoDB\Server\7.0\bin
```

Linux (Ubuntu / Debian)

Импортируйте GPG-ключ MongoDB:

```
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | sudo gpg
--dearmor -o /usr/share/keyrings/mongodb-server-7.0.gpg
```

Добавьте репозиторий:

```
echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ]
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" |
sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
```

Обновите пакеты и установите MongoDB:

```
sudo apt update

sudo apt install -y mongodb-org
```

Запустите MongoDB и добавьте в автозагрузку:

```
sudo systemctl start mongod

sudo systemctl enable mongod
```

Проверьте:

```
mongosh
```

Если видите приглашение test> — MongoDB работает.

Для остановки: sudo systemctl stop mongod

После установки на любой ОС MongoDB будет доступна по адресу mongodb://localhost:27017.

Вариант 3. Установка через Docker (рекомендуется для разработки):

Docker позволяет запустить MongoDB в изолированном контейнере без установки на основную систему. Это удобно: не засоряется ОС, легко удалить и пересоздать базу.

Убедитесь, что Docker установлен:

```
docker --version
```

Если Docker не установлен, скачайте его с https://www.docker.com/products/docker-desktop/

Скачайте официальный образ MongoDB:

```
docker pull mongo
```

Запустите контейнер MongoDB с пользователем и паролем.

MONGO_INITDB_ROOT_USERNAME и MONGO_INITDB_ROOT_PASSWORD – это переменные окружения, которые передаются в контейнер при его запуске.

```
docker run -d \
  --name mongodb \
  -p 27017:27017 \
  -e MONGO_INITDB_ROOT_USERNAME=mg_user \
  -e MONGO_INITDB_ROOT_PASSWORD=mg_pass \
  -v mongo_data:/data/db \
  mongo
```

Разбор параметров:

-d — запуск в фоновом режиме (detached).

--name mongodb — имя контейнера (чтобы обращаться по имени, а не по ID).

-p 27017:27017 — проброс порта: порт 27017 контейнера доступен на localhost:27017.

-e MONGO_INITDB_ROOT_USERNAME=mg_user — имя пользователя-администратора.

-e MONGO_INITDB_ROOT_PASSWORD=mg_pass — пароль администратора.

-v mongo_data:/data/db — именованный том Docker для хранения данных (данные сохраняются при перезапуске контейнера).

Проверьте, что контейнер запущен:

```
docker ps
```

Вы должны увидеть контейнер с именем mongodb и статусом Up.

Строка подключения для Docker-контейнера с авторизацией:

```
mongodb://mg_user:mg_pass@localhost:27017/myapp?authSource=admin
```

Параметр authSource=admin указывает, что пользователь mg_user создан в базе admin (это стандартное поведение MONGO_INITDB_ROOT_USERNAME).

В config/db.js или переменной окружения будете использовать эту строку (это мы рассмотрим далее):

```
const MONGO_URI = process.env.MONGO_URI ||
'mongodb://mg_user:mg_pass@localhost:27017/myapp?authSource=admin';
```

Полезные команды Docker для работы с MongoDB:

```
docker stop mongodb # остановить контейнер
docker start mongodb # запустить снова
docker logs mongodb # посмотреть логи MongoDB
docker exec -it mongodb mongosh -u mg_user -p mg_pass  # войти в
mongosh внутри контейнера
docker rm -f mongodb # удалить контейнер (данные сохранятся в томе
mongo_data)
docker volume rm mongo_data # удалить том с данными (необратимо!)
```

Авторизуемся в контейнер выполнив

```
docker exec -it mongodb mongosh -u mg_user -p mg_pass
```

авторизовавшись, увидите что-то вроде этого. Основная информация о сервере MongoDB, версия и т.п.

```
alex@Alexanders-MacBook-Pro Module06 % docker exec -it mongodb mongosh -u mg_user -p mg_pass
Current Mongosh Log ID: 69b676268cdf3b2573591666
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-15T09:03:34.092+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
   2026-03-15T09:03:34.304+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2026-03-15T09:03:34.305+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-03-15T09:03:34.305+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
------

test>
```

Теперь попробуем базовые команды

show dbs	Показать все базы данных
use myapp	Переключиться на базу myapp (создаётся автоматически)
db.users.insertOne({ name: "Тест", email: "test@test.com" })	Вставка документа в виде JSON объекта со свойствами name и email
db.users.find()	Показать все документы в коллекции users
db.users.find().pretty()	То же, но с форматированием
db.users.deleteMany({})	Удалить все документы
db.users.deleteOne({ _id: ObjectId('69b677b78cdf3b257359166c') })	Удалить документ из коллекции users с id='69b677b78cdf3b257359166c'
exit	Выйти из mongosh

Каждый документ автоматически получает поле _id — уникальный идентификатор типа ObjectId. Его не нужно создавать вручную.

6.4 Mongoose — ODM для MongoDB

Mongoose — это ODM (Object Document Mapper) для MongoDB и Node.js. Если по-русски, то оболочка, надстройка, интерфейс, можно называть его как угодно.

Он предоставляет строгую схему поверх гибкой MongoDB, валидацию данных, middleware (aka хуки) и удобный API для запросов.

Зачем Mongoose, если MongoDB и так гибкая? Потому что в реальных приложениях нужна предсказуемость: вы хотите быть уверены, что у каждого пользователя есть email, что возраст — число, а роль — одна из допустимых. Mongoose обеспечивает эту предсказуемость.

Что такое ODM в целом?

ODM (Object Document Mapper) — это библиотека, которая позволяет работать с документами MongoDB как с обычными JavaScript-объектами. Вместо того чтобы писать низкоуровневые запросы через нативный драйвер MongoDB, вы описываете модели с полями и типами, а Mongoose берёт на себя валидацию, преобразование типов и формирование запросов.

Аналогия: ODM для MongoDB — это как ORM (популярные библиотеки Sequelize, TypeORM) для SQL-баз. Разница в том, что ORM преобразует объекты в строки таблиц, а ODM — на документы коллекций.

Нативный драйвер (пакет mongodb) позволяет делать всё то же самое, но без удобств:

- Нет схемы — можно случайно записать { nmae: "Алексей" } вместо { name: "Алексей" }, и ошибку никто не поймает.
- Нет валидации — нужно вручную проверять каждое поле.
- Нет middleware — нельзя автоматически хешировать пароль перед сохранением.
- Нет удобных методов вроде populate() для подгрузки связанных данных.

Mongoose добавляет всё это, оставаясь тонкой прослойкой над нативным драйвером.

Установка Mongoose и подключение к MongoDB

Убедитесь, что у вас есть проект с package.json и включён ESM ("type": "module").

Установите пакет Mongoose:

```
npm install mongoose
```

Создайте файл config/db.js для подключения к базе данных:

```
import mongoose from 'mongoose';
const MONGO_URI = process.env.MONGO_URI ||
'mongodb://mg_user:mg_pass@localhost:27017/myapp?authSource=admin';

export async function connectDB() {
    try {
        await mongoose.connect(MONGO_URI);
        console.log('MongoDB подключена:', mongoose.connection.name);
    } catch (err) {
        console.error('Ошибка подключения к MongoDB:', err.message);
        process.exit(1);
    }
}
```

Подключите файл db.js в server.js и вызовите connectDB() перед запуском сервера:

```
import express from 'express';
import { connectDB } from './config/db.js';

const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3000;

connectDB().then(() => {
    app.listen(PORT, () => {
        console.log(`Сервер: http://localhost:${PORT}`);
    });
});
```

Важно: сервер запускается только после успешного подключения к БД. Если MongoDB недоступна, приложение завершится с ошибкой (process.exit(1)), а не будет работать без базы.

Теперь надо установить express в проект

```
npm install express
```

Запустите сервер и проверьте:

```
node server.js
```

В консоли должно появиться:

```
MongoDB подключена: myapp
Сервер: http://localhost:3000
```

В браузере при открытии http://localhost:3000/ должно появиться:

Cannot GET /

Если вы используете MongoDB Atlas, передайте строку подключения через переменную окружения:

```
MONGO_URI="mongodb+srv://user:pass@cluster0.xxxxx.mongodb.net/myapp"
node server.js
```

Обработка разрыва соединения

В продакшене соединение с БД может оборваться. Mongoose автоматически пытается переподключиться, но полезно логировать эти события:

```
mongoose.connection.on('error', (err) => {
    console.error('Ошибка MongoDB:', err.message);
});
mongoose.connection.on('disconnected', () => {
    console.warn('MongoDB отключена');
});
```

Соответственно наш server.js может выглядеть так:

```
import express from 'express';
import mongoose from 'mongoose';
import { connectDB } from './config/db.js';

const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3000;

connectDB().then(() => {
```

```
    app.listen(PORT, () => {
        console.log(`Сервер: http://localhost:${PORT}`);
    });
});

mongoose.connection.on('error', (err) => {
    console.error('Ошибка MongoDB:', err.message);
});
mongoose.connection.on('disconnected', () => {
    console.warn('MongoDB отключена');
});
```

6.5 Схемы и модели в Mongoose

Схема (Schema) определяет структуру документа: какие поля, какого типа, обязательные ли они, значения по умолчанию, валидация. Модель (Model) — это класс, созданный из схемы, который предоставляет методы для работы с коллекцией.

Доступные типы полей:

- String,
- Number,
- Boolean,
- Date,
- Buffer,
- ObjectId,
- Array,
- Map,
- Schema.Types.Mixed.

Пример схемы, описывающей модель User

```
import mongoose from 'mongoose';


const userSchema = new mongoose.Schema({
    name: {
        type: String,
```

```
        required: [true, 'Имя обязательно'], // [флаг, сообщение об
ошибке]
        trim: true, // Удалить пробелы по краям
        minlength: [2, 'Минимум 2 символа'],
        maxlength: [50, 'Максимум 50 символов'],
    },
    email: {
        type: String,
        required: [true, 'Email обязателен],
        unique: true, // Уникальный индекс
        lowercase: true, // Привести к нижнему регистру
        match: [/\S+@\S+\.\S+/, 'Некорректный email'],
    },
    age: {
        type: Number,
        min: [0, 'Возраст не может быть отрицательным'],
        max: 150,
    },
    role: {
        type: String,
        enum: ['user', 'admin', 'moderator'], // Допустимые значения
        default: 'user',
    },
    isActive: {
        type: Boolean,
        default: true,
    },
    tags: [String], // Массив строк
    address: { // Вложенный объект
        city: String,
        country: String,
    },
}, {
    timestamps: true, // Автоматически добавляет createdAt и updatedAt
});


// Создаём модель из схемы
// 'User' — имя модели, mongoose создаст коллекцию 'users' (lowercase +
plural)
export const User = mongoose.model('User', userSchema);
```

Опция timestamps: true автоматически добавляет поля createdAt и updatedAt к каждому документу и обновляет updatedAt при каждом изменении.

Что такое схема (Schema)?

Схема — это описание структуры документа: какие поля в нём есть, какого они типа, обязательны ли, какие значения допустимы. Схема НЕ создаёт коллекцию в MongoDB — она лишь задаёт правила.

Что такое модель (Model)?

Модель — это класс, созданный на основе схемы. Именно модель предоставляет методы для работы с коллекцией: create(), find(), findById(), findByIdAndUpdate(), findByIdAndDelete() и т.д. Одна модель = одна коллекция.

Давайте создадим схему модели User

Создайте файл models/User.js и с содержимым:

```
import mongoose from 'mongoose';
const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Имя обязательно'],
        trim: true,
        minlength: [2, 'Минимум 2 символа'],
    },
    email: {
        type: String,
        required: [true, 'Email обязателен'],
        unique: true,
        lowercase: true,
    },
    role: {
        type: String,
        enum: ['user', 'admin', 'moderator'],
        default: 'user',
    },
}, {
    timestamps: true,
});

export const User = mongoose.model('User', userSchema);
```

Схема указывает на типы полей (type: String) и имеет валидаторы данных (required, trim, minlength и т.п.)

Разбор каждого валидатора в примере:

Валидатор	Что делает?
required: [true, "Имя обязательно"]	Поле обязательно для заполнения. Если не передать — Mongoose выбросит ValidationError с сообщением "Имя обязательно".
trim: true	Автоматически удаляет пробелы в начале и конце строки. " Алексей " станет "Алексей".
minlength / maxlength	Минимальная и максимальная длина строки. Работает только для типа String.
unique: true	Создаёт уникальный индекс в MongoDB. Если попытаться создать второго пользователя с тем же email — MongoDB вернёт ошибку с кодом 11000 (duplicate key). Важно: unique — это НЕ валидатор Mongoose, а индекс MongoDB. Он не выбрасывает ValidationError.
lowercase: true	Автоматически приводит значение к нижнему регистру. "Alex@Example.COM" станет "alex@example.com".
enum: ["user", "admin", "moderator"]	Значение должно быть одним из перечисленных. Если передать "superadmin" — Mongoose выбросит ValidationError.
default: "user"	Если поле не передано при создании документа, Mongoose подставит это значение автоматически.
timestamps: true	Опция схемы (не поля). Автоматически добавляет два поля: createdAt (дата создания) и updatedAt (дата последнего изменения). Mongoose обновляет updatedAt при каждом сохранении.

Теперь проверим наши валидаторы. Создайте файл test-model.js:

```
import mongoose from 'mongoose';
import { User } from './models/User.js';
```

```
await
mongoose.connect('mongodb://mg_user:mg_pass@localhost:27017/myapp?authS
ource=admin');

// Попробуем создать пользователя без обязательных полей
try {
    await User.create({ name: "A" });
} catch (err) {
    console.log('Ошибка валидации:', err.message);
}

// Создаём корректного пользователя
const user = await User.create({
    name: 'Алексей',
    email: 'ALEX@Example.COM',
});
console.log('Создан:', user);
console.log('Email стал:', user.email); // alex@example.com (lowercase)
console.log('Role:', user.role);         // user (default)
console.log('createdAt:', user.createdAt); // дата создания

await mongoose.disconnect();
```

Запустите:

```
node test-model.js
```

Вы увидите, что:

- Попытка создать пользователя с именем из 1 символа вызовет ошибку валидации.
- Email автоматически приведён к нижнему регистру.
- Роль автоматически установлена в "user".
- Появились поля createdAt и updatedAt.

```
alex@Alexanders-MacBook-Pro Module06 % node test-model.js
Ошибка валидации: User validation failed: email: Email обязателен, name: Минимум 2 символа
Создан: {
  name: 'Алексей',
  email: 'alex@example.com',
  role: 'user',
  _id: new ObjectId('69b683451af352a51b2529f4'),
  createdAt: 2026-03-15T10:00:37.802Z,
  updatedAt: 2026-03-15T10:00:37.802Z,
  __v: 0
}
Email стал: alex@example.com
Role: user
createdAt: 2026-03-15T10:00:37.802Z
```

6.6 CRUD-операции с Mongoose

Mongoose предоставляет удобные методы для всех операций с данными:

Создание (Create):

```
const user = await User.create({
    name: 'Алексей',
    email: 'alex@example.com',
    age: 25,
});
// Mongoose проверит данные по схеме и выбросит ValidationError при
ошибке
```

Чтение (Read):

```
const all = await User.find();                          // Все документы
const active = await User.find({ isActive: true });    // С фильтром
const one = await User.findById('64a...');              // По ID
const byEmail = await User.findOne({ email: 'alex@example.com' });
```

Методы find() и findOne() возвращают Query-объект, к которому можно добавлять цепочку методов для сортировки, пагинации и выбора полей:

```
const users = await User.find({ role: 'user' })
    .sort({ createdAt: -1 })     // -1 = по убыванию (новые первые)
    .skip(20)                     // Пропустить 20 документов
(пагинация)
    .limit(10)                    // Взять 10 документов
    .select('name email -_id');   // Только name и email, без _id
```

Обновление (Update):

```
// findByIdAndUpdate возвращает обновлённый документ (с опцией new:
true)
const updated = await User.findByIdAndUpdate(
    id,
    { name: 'Новое имя' },
    { new: true, runValidators: true }  // new: вернуть обновлённый;
runValidators: проверить схему
);
```

Опция runValidators: true важна — без неё Mongoose не проверяет данные при обновлении.

Удаление (Delete):

```
await User.findByIdAndDelete(id);
await User.deleteMany({ isActive: false });  // Удалить все неактивные
```

Добавляем CRUD-операции в наше приложение

Теперь, когда у нас есть модель User и подключение к MongoDB, создадим Express-маршруты для каждой операции. Создайте файл routes/users.js:

```
import { Router } from 'express';
import { User } from '../models/User.js';


const router = Router();
```

и начинайте добавление маршрутов всех CRUD-операций в users.js

Create — создание нового пользователя

Для создания документа используем метод Model.create(). Он принимает объект с данными, проверяет их по схеме и сохраняет в коллекцию. Если валидация не пройдёт — Mongoose выбросит ValidationError.

Маршрут POST /api/users:

```
router.post('/', async (req, res, next) => {
    try {
        const user = await User.create(req.body);
        res.status(201).json(user);
    } catch (err) {
        if (err.name === 'ValidationError') {
            return res.status(400).json({ error: err.message });
        }
        if (err.code === 11000) {
            return res.status(409).json({ error: 'Такой email уже
существует' });
        }
        next(err);
    }
});
```

Что здесь происходит:

- req.body содержит данные из тела запроса (JSON). Для этого в server.js должен быть app.use(express.json()).
- User.create(req.body) создаёт документ и сохраняет его в коллекцию users.

- Статус 201 (Created) — стандартный ответ при успешном создании ресурса.
- Если Mongoose не пропустит данные по схеме — ловим ValidationError и возвращаем 400 (Bad Request).
- Если email не уникален — MongoDB вернёт ошибку с кодом 11000, возвращаем 409 (Conflict).

Read — получение списка пользователей

Для чтения данных используем Model.find(). Без аргументов — вернёт все документы. С объектом-фильтром — только подходящие. Результат — массив документов.

Маршрут GET /api/users с пагинацией и фильтрацией:

```
router.get('/', async (req, res, next) => {
    try {
        const { page = 1, limit = 10, role } = req.query;
        const filter = role ? { role } : {};

        const users = await User.find(filter)
            .sort({ createdAt: -1 })
            .skip((page - 1) * limit)
            .limit(Number(limit))
            .select('name email role createdAt');

        const total = await User.countDocuments(filter);

        res.json({
            users,
            total,
            page: Number(page),
            pages: Math.ceil(total / limit),
        });
    } catch (err) {
        next(err);
    }
});
```

Разбор Query-цепочки:

- .sort({ createdAt: -1 }) — сортировка по дате создания, новые первые (-1 означает по убыванию).
- .skip((page - 1) * limit) — пропустить документы предыдущих страниц. Для page=2 и limit=10 пропустим первые 10.
- .limit(Number(limit)) — взять не больше limit документов.
- .select('name email role createdAt') — вернуть только указанные поля. Можно исключить поле: '-password'.
- countDocuments(filter) — общее количество документов для пагинации.

Read — получение одного пользователя по ID

Model.findById(id) ищет документ по полю _id. Если документ не найден — возвращает null.

Маршрут GET /api/users/:id:

```
router.get('/:id', async (req, res, next) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) {
            return res.status(404).json({ error: 'Пользователь не
найден' });
        }
        res.json(user);
    } catch (err) {
        next(err);
    }
});
```

Если передать невалидный формат ID (не 24 hex-символа) — Mongoose выбросит CastError. Он попадёт в next(err) и обработается централизованным обработчиком ошибок.

Update — обновление пользователя

Model.findByIdAndUpdate(id, data, options) находит документ по ID и обновляет его. Важные опции:

- new: true — вернуть обновлённый документ (без этой опции возвращается старая версия).

- runValidators: true — проверить новые данные по схеме. По умолчанию Mongoose НЕ валидирует данные при update!

Маршрут PATCH /api/users/:id (частичное обновление):

```
router.patch('/:id', async (req, res, next) => {
    try {
        const user = await User.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true, runValidators: true }
        );
        if (!user) {
            return res.status(404).json({ error: 'Пользователь не
найден' });
        }
        res.json(user);
    } catch (err) {
        next(err);
    }
});
```

Почему PATCH, а не PUT?

- PATCH — частичное обновление: отправляем только изменённые поля.
- PUT — полная замена: нужно отправить весь объект целиком.

В большинстве API используют PATCH, потому что он удобнее для клиента.

Если по секрету (только тссс!! никому не говорите) можно даже использовать обычный POST, но исходя из правил хорошего тона и лучших практик программирования рекомендуется использовать PATCH или PUT при обновлении, хотя по сути они тот же POST.

Delete — удаление пользователя

Model.findByIdAndDelete(id) находит документ по ID и удаляет его из коллекции. Возвращает удалённый документ или null, если не найден.

Маршрут DELETE /api/users/:id:

```
router.delete('/:id', async (req, res, next) => {
    try {
        const user = await User.findByIdAndDelete(req.params.id);
        if (!user) {
            return res.status(404).json({ error: 'Пользователь не
найден' });
        }
        res.status(204).end();
    } catch (err) {
        next(err);
    }
});
```

Статус 204 (No Content) — стандартный ответ при успешном удалении. Тело ответа пустое — клиенту не нужно получать данные удалённого объекта.

Экспорт роутера и подключение в server.js

В конце файла routes/users.js экспортируйте роутер:

```
export default router;
```

И подключите в server.js:

```
import express from 'express';
import mongoose from 'mongoose';
import { connectDB } from './config/db.js';
import usersRouter from './routes/users.js';

const app = express();
app.use(express.json());
app.use('/api/users', usersRouter);

const PORT = process.env.PORT || 3000;

connectDB().then(() => {
    app.listen(PORT, () => {
```

```
        console.log(`Сервер: http://localhost:${PORT}`);
    });
});

mongoose.connection.on('error', (err) => {
    console.error('Ошибка MongoDB:', err.message);
});
mongoose.connection.on('disconnected', () => {
    console.warn('MongoDB отключена');
});
```

Теперь все маршруты доступны по префиксу /api/users:

- POST /api/users — создать пользователя
- GET /api/users — получить список
- GET /api/users/:id — получить по ID
- PATCH /api/users/:id — обновить
- DELETE /api/users/:id — удалить

Теперь проверим все маршруты. Запустим сервер и проверим маршруты.

Создание

```
curl -X POST http://localhost:3000/api/users \
  -H 'Content-Type: application/json' \
  -d '{"name": "Алексей", "email": "alex@example.com"}'
```

Ожидаемый ответ (статус 201):

```
{
    "_id": "64a1b2c3d4e5f6a7b8c9d0e1",
    "name": "Алексей",
    "email": "alex@example.com",
    "role": "user",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

На Windows может возникать проблема, что json не парсится. Поэтому нужно обернуть payload всех POST/PUT/PATCH запросов, а также

Content-Type: application/json в двойные кавычки вместо одиночных. Пример:

```
curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"Алексей\",\"email\":\"alex@example.com\"}"
```

Попробуйте отправить некорректные данные и убедитесь, что валидация работает:

```
curl -X POST http://localhost:3000/api/users \
  -H 'Content-Type: application/json' \
  -d '{"name": "A"}'
```

Ответ: 400, сообщение об ошибке валидации (имя слишком короткое, email обязателен).

Чтение всех пользователей

```
curl http://localhost:3000/api/users
curl "http://localhost:3000/api/users?page=1&limit=5"
```

Чтение одного пользователя по ID

Подставьте реальный _id из ответа на создание или чтение всех вместо <_id> :

```
curl http://localhost:3000/api/users/<_id>
```

Обновление пользователя

Подставьте реальный _id вместо <_id> :

```
curl -X PATCH http://localhost:3000/api/users/<_id> \
  -H 'Content-Type: application/json' \
  -d '{"name": "Новое имя"}'
```

В ответе вернётся обновлённый пользователь с новым именем и обновлённым полем updatedAt.

Удаление

Подставьте реальный _id вместо <_id> :

```
curl -X DELETE http://localhost:3000/api/users/<_id>
```

Ответ: пустое тело, статус 204. Повторный запрос на тот же ID вернёт 404.

6.7 Связи между моделями (populate)

В MongoDB нет JOIN-запросов как в SQL. Вместо этого Mongoose использует ссылки (references) и метод populate() для подгрузки связанных данных.

```
// models/Post.js — пост ссылается на автора (User)
const postSchema = new mongoose.Schema({
    title: { type: String, required: true },
    content: String,
    author: {
        type: mongoose.Schema.Types.ObjectId, // Хранит ID пользователя
        ref: 'User',                          // Ссылка на модель User
        required: true,
    },
    comments: [{
        text: String,
        user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
        createdAt: { type: Date, default: Date.now },
    }],
}, { timestamps: true });


export const Post = mongoose.model('Post', postSchema);
```

Метод populate() заменяет ObjectId на полный документ из связанной коллекции:

```
// Без populate: author = '64a1b2c3d4e5f6...'
// С populate: author = { _id: '64a...', name: 'Алексей', email:
'alex@...' }
const posts = await Post.find()
    .populate('author', 'name email')   // Подгрузить автора (только
name и email)
    .populate('comments.user', 'name'); // Подгрузить авторов
комментариев
```

Зачем нужны связи в MongoDB?

В реальном приложении данные связаны между собой: у поста есть автор, у заказа — покупатель и товары, у комментария — пользователь.

В SQL-базах для этого используется JOIN. В MongoDB нет JOIN, но есть два подхода:

1) Встраивание (embedding)

Данные вкладываются прямо в документ. Например, адрес внутри пользователя:

```
{
    name: "Алексей",
    address: {
        city: "Москва",
        street: "Ленина 10"
    }
}
```

Плюсы: один запрос — все данные; нет дополнительных запросов к БД.

Минусы: если вложенный объект большой или дублируется между документами — данные раздуваются.

2) Ссылки (references)

Документ хранит только ObjectId связанного документа. Данные подгружаются отдельным запросом через populate().

```
{
    title: "Мой пост",
    author: ObjectId("64a1b2c3d4e5f6a7b8c9d0e1")  // только ID
}
```

Плюсы: нет дублирования данных; каждый объект хранится один раз.

Минусы: нужен дополнительный запрос (populate) для подгрузки связанных данных.

Когда встраивать, а когда ссылаться?

Встраивание подходит, когда:

- Данные всегда используются вместе (адрес пользователя).
- Вложенный объект не растёт бесконечно.
- Данные не дублируются между документами.

Ссылки подходят, когда:

- Данные используются независимо (пользователь и его посты).
- Один объект связан со многими другими (автор → много постов).
- Связанные данные часто обновляются (изменение имени автора не должно требовать обновления всех постов).

Создаём модель Post со ссылкой на User

Создайте файл models/Post.js:

```
import mongoose from 'mongoose';

const postSchema = new mongoose.Schema({
    title: {
        type: String,
        required: [true, 'Заголовок обязателен'],
    },
    content: String,
    author: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true,
    },
}, { timestamps: true });

export const Post = mongoose.model('Post', postSchema);
```

Разбор поля author:

- type: mongoose.Schema.Types.ObjectId — поле хранит не объект, а только 24-символьный идентификатор.

- ref: 'User' — указывает Mongoose, из какой модели (коллекции) подгружать данные при вызове populate().
- required: true — пост не может существовать без автора.

Как работает populate()

Без populate() поле author содержит просто строку ObjectId:

```
const post = await Post.findById(postId);
console.log(post.author);
// "64a1b2c3d4e5f6a7b8c9d0e1"  ← просто ID, не объект
```

С populate() Mongoose делает дополнительный запрос к коллекции users и заменяет ObjectId на полный объект:

```
const post = await Post.findById(postId).populate('author', 'name
email');
console.log(post.author);
// { _id: "64a...", name: "Алексей", email: "alex@example.com" }  ←
объект!
```

Второй аргумент populate — строка с именами полей, которые нужно подгрузить. Если не указать — загрузятся все поля связанного документа (включая, например, пароль). Поэтому всегда указывайте нужные поля.

Тестируем связи. Создайте файл test-populate.js:

```
import mongoose from 'mongoose';
import { User } from './models/User.js';
import { Post } from './models/Post.js';


await
mongoose.connect('mongodb://mg_user:mg_pass@localhost:27017/myapp?authS
ource=admin');


// Шаг 1. Создаём пользователя
const user = await User.create({
    name: 'Алексей',
    email: 'alex-populate-test@example.com',
});
console.log('Создан пользователь:', user._id);
```

```
// Шаг 2. Создаём пост, указывая author = _id пользователя
const post = await Post.create({
    title: 'Мой первый пост',
    content: 'Привет, мир!',
    author: user._id,
});
console.log('Пост (без populate):');
console.log('  post.author =', post.author);  // ObjectId

// Шаг 3. Читаем пост с populate
const postWithAuthor = await Post.findById(post._id)
    .populate('author', 'name email');

console.log('Пост (с populate):');
console.log('  post.author =', postWithAuthor.author);
// { _id: "...", name: "Алексей", email: "alex-populate-
test@example.com" }

await mongoose.disconnect();
```

Запустите:

```
node test-populate.js
```

Вы увидите разницу:

- Без populate — author содержит строку ObjectId.
- С populate — author заменяется на объект пользователя с полями name и email.

Использование populate() в Express-маршрутах

Создайте файл routes/posts.js:

```
import { Router } from 'express';
import { Post } from '../models/Post.js';

const router = Router();

// Создание поста
router.post('/', async (req, res, next) => {
    try {
        const post = await Post.create(req.body);
        res.status(201).json(post);
    } catch (err) {
        next(err);
```

```
    }
});

// Получение всех постов с данными авторов
router.get('/', async (req, res, next) => {
    try {
        const posts = await Post.find()
            .sort({ createdAt: -1 })
            .populate('author', 'name email');
        res.json(posts);
    } catch (err) {
        next(err);
    }
});

// Получение поста по ID
router.get('/:id', async (req, res, next) => {
    try {
        const post = await Post.findById(req.params.id)
            .populate('author', 'name email');
        if (!post) {
            return res.status(404).json({ error: 'Пост не найден' });
        }
        res.json(post);
    } catch (err) {
        next(err);
    }
});

export default router;
```

Подключите в server.js:

```
import postsRouter from './routes/posts.js';
app.use('/api/posts', postsRouter);
```

Создание поста (укажите реальный _id пользователя в поле author):

```
curl -X POST http://localhost:3000/api/posts \
  -H 'Content-Type: application/json' \
  -d '{"title": "Мой пост", "content": "Текст поста", "author":
"64a1b2c3d4e5f6a7b8c9d0e1"}'
```

Получение всех постов (author будет объектом с name и email):

```
curl http://localhost:3000/api/posts
```

Ожидаемый ответ:

```
[
    {
        "_id": "...",
        "title": "Мой пост",
        "content": "Текст поста",
        "author": {
            "_id": "64a1b2c3d4e5f6a7b8c9d0e1",
            "name": "Алексей",
            "email": "alex@example.com"
        },
        "createdAt": "...",
        "updatedAt": "..."
    }
]
```

Множественный populate

Если в схеме несколько полей-ссылок, вызовите populate() для каждого:

```
const posts = await Post.find()
    .populate('author', 'name email')
    .populate('comments.user', 'name');
```

Каждый вызов populate() — это отдельный запрос к MongoDB. Не злоупотребляйте: если подгружаете 5 связей для 100 постов — это 5 дополнительных запросов.

6.8 Фильтрация, поиск и агрегация

Mongoose предоставляет мощные инструменты для фильтрации, поиска и агрегации данных. В этом разделе мы изучим операторы сравнения, логические операторы, регулярные выражения для поиска, сортировку, пагинацию и выборку полей.

Операторы сравнения

MongoDB поддерживает операторы сравнения для фильтрации документов. В Mongoose они передаются как объекты внутри условия where:

```
// Книги с количеством страниц больше 200
const books = await Post.find({ pages: { $gt: 200 } });

// Пользователи с возрастом от 18 до 30
const users = await User.find({ age: { $gte: 18, $lte: 30 } });

// Пользователи с ролью admin или moderator
const admins = await User.find({ role: { $in: ['admin', 'moderator'] }
});
```

Основные операторы сравнения:

- $gt — больше (greater than).

- $gte — больше или равно (greater than or equal).

- $lt — меньше (less than).

- $lte — меньше или равно (less than or equal).

- $eq — равно (обычно не нужен, можно писать просто { field: value }).

- $ne — не равно (not equal).

- $in — значение входит в массив.

- $nin — значение НЕ входит в массив.

Логические операторы

Для сложных условий используйте логические операторы $and, $or и $not:

```
// Посты, у которых title содержит 'Node' ИЛИ 'Express'
const posts = await Post.find({
    $or: [
        { title: /Node/i },
        { title: /Express/i },
    ],
});
```

```
// Пользователи с ролью admin И email на домене @company.com
const companyAdmins = await User.find({
    $and: [
        { role: 'admin' },
        { email: /@company\.com$/i },
    ],
});



// Пользователи с ролью НЕ admin
const nonAdmins = await User.find({ role: { $ne: 'admin' } });
```

Если передать несколько условий в один объект find(), MongoDB неявно применяет $and:

```
// Эти два запроса эквивалентны:
await User.find({ role: 'admin', name: /Иван/i });
await User.find({ $and: [{ role: 'admin' }, { name: /Иван/i }] });
```

Поиск по тексту ($regex)

Для регистронезависимого поиска по строковым полям используйте оператор $regex с опцией $options: 'i':

```
// Поиск постов по title (регистронезависимый)
const posts = await Post.find({
    title: { $regex: 'node', $options: 'i' },
});
```

Можно также использовать встроенные регулярные выражения JavaScript:

```
// Эквивалентный запрос через RegExp
const posts = await Post.find({ title: /node/i });
```

Для поиска по нескольким полям одновременно используйте $or:

```
// Поиск по title И content
const query = 'express';
const posts = await Post.find({
    $or: [
        { title: { $regex: query, $options: 'i' } },
        { content: { $regex: query, $options: 'i' } },
    ],
});
```

Опция $options: 'i' означает case-insensitive — поиск не различает строчные и прописные буквы.

Фильтрация по массивам

MongoDB умеет искать значения внутри массивов. Если поле содержит массив (например, tags), можно искать по нему напрямую:

```
// Найти все посты с тегом 'javascript'
const posts = await Post.find({ tags: 'javascript' });
// MongoDB автоматически ищет значение в массиве

// Найти посты, у которых есть ВСЕ указанные теги
const posts2 = await Post.find({ tags: { $all: ['javascript',
'tutorial'] } });

// Получить список всех уникальных тегов
const allTags = await Post.distinct('tags');
```

Сортировка

Метод .sort() задаёт порядок результатов. Значение 1 — по возрастанию (asc), -1 — по убыванию (desc):

```
// Посты от новых к старым
const posts = await Post.find().sort({ createdAt: -1 });

// Пользователи по имени (A -> Z)
const users = await User.find().sort({ name: 1 });

// Сортировка по нескольким полям
const sorted = await Post.find().sort({ author: 1, createdAt: -1 });
```

Пагинация (skip и limit)

Для постраничного вывода используйте комбинацию .skip() и .limit():

```
const page = 2;       // Номер страницы (начиная с 1)
const limit = 10;     // Записей на страницу

const posts = await Post.find()
    .sort({ createdAt: -1 })
```

```
    .skip((page - 1) * limit)
    .limit(limit);

// Для ответа клиенту полезно знать общее количество:
const total = await Post.countDocuments(filter);
const pages = Math.ceil(total / limit);
```

Метод countDocuments(filter) возвращает количество документов, подходящих под фильтр. Для подсчёта всех документов можно передать пустой объект {}.

Для параллельного выполнения запроса данных и подсчёта используйте Promise.all():

```
const [posts, total] = await Promise.all([
    Post.find(filter).sort({ createdAt: -1 }).skip((page - 1) *
limit).limit(limit),
    Post.countDocuments(filter),
]);
```

Выборка полей (select)

Метод .select() позволяет указать, какие поля включить или исключить из результата:

```
// Включить только name и email
const users = await User.find().select('name email');

// Исключить password из результата
const users2 = await User.find().select('-password');

// Комбинация: только имя и дата последнего поста
const lastPost = await Post.findOne({ author: userId })
    .sort({ createdAt: -1 })
    .select('title createdAt');
```

Используйте .select() для оптимизации: не загружайте лишние поля, особенно если документы содержат большие текстовые поля или массивы.

Добавим поиск и фильтрацию в маршруты

Применим изученные инструменты в Express-маршрутах. Откройте файл routes/posts.js (из раздела 6.7) и добавьте маршрут поиска. Он должен быть объявлен ДО маршрута /:id, иначе Express воспримет слово 'search' как параметр id:

```
// GET /api/posts/search?q=запрос
router.get('/search', async (req, res, next) => {
    try {
        const { q } = req.query;
        if (!q) return res.status(400).json({ error: 'Параметр q
обязателен' });

        const posts = await Post.find({
            $or: [
                { title: { $regex: q, $options: 'i' } },
                { content: { $regex: q, $options: 'i' } },
            ],
        }).populate('author', 'name email');

        res.json(posts);
    } catch (err) { next(err); }
});
```

Теперь добавим фильтрацию, сортировку и пагинацию в маршрут получения всех постов:

```
// GET
/api/posts?page=1&limit=10&sort=createdAt&order=desc&author=<userId>
router.get('/', async (req, res, next) => {
    try {
        const {
            page = 1,
            limit = 10,
            sort = 'createdAt',
            order = 'desc',
            author,
        } = req.query;

        const filter = {};
        if (author) filter.author = author;

        const [posts, total] = await Promise.all([
            Post.find(filter)
                .populate('author', 'name email')
                .sort({ [sort]: order === 'asc' ? 1 : -1 })
```

```
                .skip((page - 1) * limit)
                .limit(Number(limit)),
            Post.countDocuments(filter),
        ]);

        res.json({
            posts,
            total,
            page: Number(page),
            pages: Math.ceil(total / limit),
        });
    } catch (err) { next(err); }
});
```

Тестирование

Перезапустите сервер и проверьте новые возможности:

```
# Поиск постов по слову
curl "http://localhost:3000/api/posts/search?q=express"

# Постраничный вывод
curl "http://localhost:3000/api/posts?page=1&limit=5"

# Сортировка по title (A -> Z)
curl "http://localhost:3000/api/posts?sort=title&order=asc"

# Фильтрация по автору
curl "http://localhost:3000/api/posts?author=<userId>"
```

Ответ на GET /api/posts теперь содержит объект с полями posts (массив), total (общее количество), page (текущая страница) и pages (всего страниц).

6.9 Переменные окружения в приложении

В предыдущих разделах мы указывали строку подключения к MongoDB прямо в коде. Это плохая практика: пароли и адреса серверов не должны быть в исходных файлах. Для этого существуют переменные окружения (environment variables).

Что такое переменные окружения?

Переменные окружения — это пары «ключ=значение», которые хранятся вне кода и доступны приложению через process.env. Они используются для:

- Паролей и секретов (строки подключения к БД, API-ключи, JWT-секреты).
- Настроек, зависящих от среды (порт сервера, режим debug/production).
- Адресов внешних сервисов.

В Node.js переменные окружения доступны через объект process.env:

```
console.log(process.env.PORT);       // undefined (если не задана)
console.log(process.env.NODE_ENV);   // undefined (если не задана)
```

Пакет dotenv

Пакет dotenv загружает переменные из файла .env в process.env. Это стандартный способ управления конфигурацией в Node.js-проектах.

Установка:

```
npm install dotenv
```

Создание файла .env

Создайте файл .env (обратите внимание на точку перед именем файла, она обязательна!) в корне проекта:

```
PORT=3000
MONGO_URI=mongodb://mg_user:mg_pass@localhost:27017/myapp?authSource=ad
min
NODE_ENV=development
```

Правила оформления файла .env:

- Каждая переменная на отдельной строке в формате КЛЮЧ=значение.
- Без пробелов вокруг знака =

- Значения с пробелами оборачивайте в кавычки: MY_VAR="hello world".
- Комментарии начинаются с #.
- Имена переменных — в UPPER_SNAKE_CASE (общепринятое соглашение).

Подключение dotenv

Вариант 1. Импорт в начале server.js (классический способ):

```
import 'dotenv/config';
import express from 'express';
import { connectDB } from './config/db.js';

const PORT = process.env.PORT || 3000;
console.log(process.env.MONGO_URI); // значение из .env
```

Строка import 'dotenv/config' загружает .env файл и заполняет process.env. Важно: этот импорт должен быть первым, до любых других импортов, которые используют process.env.

Вариант 2. Через флаг --env-file (Node.js 20.6+):

```
node --env-file=.env server.js
```

В этом случае пакет dotenv не нужен — Node.js сам загрузит .env файл.

Можно добавить скрипты в package.json:

```
"scripts": {
    "start": "node --env-file=.env server.js",
    "dev": "node --env-file=.env --watch server.js"
}
```

Обновляем config/db.js

Теперь строка подключения берётся из переменной окружения:

```
import mongoose from 'mongoose';

export async function connectDB() {
    const uri = process.env.MONGO_URI;
    if (!uri) {
```

```
        console.error('MONGO_URI не задана в .env');
        process.exit(1);
    }
    try {
        await mongoose.connect(uri);
        console.log('MongoDB подключена:', mongoose.connection.name);
    } catch (err) {
        console.error('Ошибка подключения к MongoDB:', err.message);
        process.exit(1);
    }
}
```

Если MONGO_URI не задана — приложение сразу завершится с понятным сообщением об ошибке, а не упадёт позже с непонятным TypeError.

Файл .env.example

Файл .env содержит реальные пароли и не должен попадать в git. Вместо него в репозиторий добавляют .env.example — шаблон с пустыми значениями:

```
# .env.example — скопируйте в .env и заполните
PORT=3000
MONGO_URI=
NODE_ENV=development
```

Использование переменных окружения в приложении при подключении с помощью варианта 1:

Обновлённый server.js:

```
import 'dotenv/config'; // ← подключает .env
import express from 'express';
import { connectDB } from './config/db.js';
import usersRouter from './routes/users.js';
import postsRouter from './routes/posts.js';

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.use('/api/users', usersRouter);
```

```
app.use('/api/posts', postsRouter);

connectDB().then(() => {
    app.listen(PORT, () => {
        console.log(`Сервер: http://localhost:${PORT}`);
    });
});
```

Теперь все секреты и настройки хранятся в .env, а код не содержит захардкоженных значений. Это важно для безопасности и удобства: при деплое на сервер достаточно задать переменные окружения, не меняя код.

6.10 Express + Mongoose: полноценное приложение

Собираем проект целиком

В этом разделе мы пошагово соберём полноценное Express-приложение с MongoDB: от создания проекта до тестирования всех эндпоинтов.

Структура проекта

```
my-express-mongo/
├── config/
│   └── db.js           # Подключение к MongoDB
├── models/
│   ├── User.js         # Модель пользователя
│   └── Post.js         # Модель поста
├── routes/
│   ├── users.js        # CRUD-маршруты для пользователей
│   └── posts.js        # CRUD-маршруты для постов
├── server.js           # Главный файл приложения
├── .env                # Переменные окружения
└── package.json
```

Шаг 1. Создайте проект

```
mkdir my-express-mongo
cd my-express-mongo
npm init -y
```

Откройте package.json и добавьте "type": "module" для поддержки ESM модулей:

```
{
    "name": "my-express-mongo",
    "type": "module",
    "scripts": {
        "start": "node --env-file=.env server.js",
        "dev": "node --env-file=.env --watch server.js"
    }
}
```

Скрипт "dev" использует встроенный --watch (Node.js 18+) для автоматической перезагрузки при изменении файлов.

Шаг 2. Установите зависимости

```
npm install express mongoose
```

Шаг 3. Создайте config/db.js и .env

Файл отвечает за подключение к MongoDB. Если подключение не удалось — приложение завершается:

```
import mongoose from 'mongoose';

const MONGO_URI = process.env.MONGO_URI;

export async function connectDB() {
    try {
        await mongoose.connect(MONGO_URI);
        console.log('MongoDB подключена:', mongoose.connection.name);
    } catch (err) {
        console.error('Ошибка подключения к MongoDB:', err.message);
        process.exit(1);
    }
}
```

Файл .env

```
PORT=3000
MONGO_URI=mongodb://mg_user:mg_pass@localhost:27017/myapp?authSource=ad
min
NODE_ENV=development
```

Шаг 4. Создайте модели

models/User.js (из раздела 6.5 «Схемы и модели»):

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Имя обязательно'],
        trim: true,
        minlength: [2, 'Минимум 2 символа'],
    },
    email: {
        type: String,
        required: [true, 'Email обязателен'],
        unique: true,
        lowercase: true,
    },
    role: {
        type: String,
        enum: ['user', 'admin', 'moderator'],
        default: 'user',
    },
}, { timestamps: true });

export const User = mongoose.model('User', userSchema);
```

models/Post.js (из раздела 6.7 «Связи между моделями»):

```
import mongoose from 'mongoose';

const postSchema = new mongoose.Schema({
    title: {
        type: String,
        required: [true, 'Заголовок обязателен'],
    },
    content: String,
    author: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true,
    },
}, { timestamps: true });

export const Post = mongoose.model('Post', postSchema);
```

Шаг 5. Создайте роутеры

routes/users.js — полный CRUD для пользователей (из раздела 6.6 «CRUD-операции»):

```
import { Router } from 'express';
import { User } from '../models/User.js';

const router = Router();

// Создать
router.post('/', async (req, res, next) => {
    try {
        const user = await User.create(req.body);
        res.status(201).json(user);
    } catch (err) {
        if (err.name === 'ValidationError') {
            return res.status(400).json({ error: err.message });
        }
        if (err.code === 11000) {
            return res.status(409).json({ error: 'Email уже существует'
});
        }
        next(err);
    }
});

// Читать всех
router.get('/', async (req, res, next) => {
    try {
        const { page = 1, limit = 10, role } = req.query;
        const filter = role ? { role } : {};

        const [users, total] = await Promise.all([
            User.find(filter)
                .skip((page - 1) * limit)
                .limit(Number(limit))
                .sort({ createdAt: -1 }),
            User.countDocuments(filter),
        ]);

        res.json({ users, total, page: Number(page), pages:
Math.ceil(total / limit) });
    } catch (err) { next(err); }
});

// Читать одного
```

```
router.get('/:id', async (req, res, next) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) return res.status(404).json({ error: 'Пользователь
не найден' });
        res.json(user);
    } catch (err) { next(err); }
});

// Обновить
router.patch('/:id', async (req, res, next) => {
    try {
        const user = await User.findByIdAndUpdate(
            req.params.id, req.body, { new: true, runValidators: true }
        );
        if (!user) return res.status(404).json({ error: 'Пользователь
не найден' });
        res.json(user);
    } catch (err) { next(err); }
});

// Удалить
router.delete('/:id', async (req, res, next) => {
    try {
        const user = await User.findByIdAndDelete(req.params.id);
        if (!user) return res.status(404).json({ error: 'Пользователь
не найден' });
        res.status(204).end();
    } catch (err) { next(err); }
});

export default router;
```

routes/posts.js — маршруты для постов с populate (из раздела 6.7 «Связи между моделями»):

```
import { Router } from 'express';
import { Post } from '../models/Post.js';

const router = Router();

router.post('/', async (req, res, next) => {
    try {
        const post = await Post.create(req.body);
        res.status(201).json(post);
    } catch (err) { next(err); }
```

```
});

router.get('/', async (req, res, next) => {
    try {
        const posts = await Post.find()
            .sort({ createdAt: -1 })
            .populate('author', 'name email');
        res.json(posts);
    } catch (err) { next(err); }
});

router.get('/:id', async (req, res, next) => {
    try {
        const post = await
Post.findById(req.params.id).populate('author', 'name email');
        if (!post) return res.status(404).json({ error: 'Пост не
найден' });
        res.json(post);
    } catch (err) { next(err); }
});

export default router;
```

Шаг 6. Создайте server.js — главный файл приложения

```
import express from 'express';
import { connectDB } from './config/db.js';
import usersRouter from './routes/users.js';
import postsRouter from './routes/posts.js';

const app = express();
const PORT = process.env.PORT || 3000;

// Парсинг JSON-тела запроса
app.use(express.json());

// Логирование каждого запроса
app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} ${req.method}
${req.url}`);
    next();
});

// Маршруты
app.use('/api/users', usersRouter);
app.use('/api/posts', postsRouter);
```

```
// 404 — маршрут не найден
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});

// Централизованная обработка ошибок
app.use((err, req, res, next) => {
    console.error(err.stack);
    const status = err.status || 500;
    res.status(status).json({ error: err.message || 'Внутренняя ошибка
сервера' });
});

// Подключаемся к БД, затем запускаем сервер
connectDB().then(() => {
    app.listen(PORT, () => {
        console.log(`Сервер запущен: http://localhost:${PORT}`);
    });
});
```

Порядок middleware в server.js важен:

1. express.json() — парсинг тела запроса (должен быть первым).
2. Логгер — логирует все запросы.
3. Роутеры — обрабатывают маршруты /api/users и /api/posts.
4. 404-обработчик — ловит все запросы, которые не попали ни в один роутер.
5. Обработчик ошибок (4 аргумента) — ловит все ошибки, переданные через next(err).

Шаг 7. Запустите сервер

```
npm run dev
```

В консоли должно появиться:

```
MongoDB подключена: myapp
Сервер запущен: http://localhost:3000
```

Шаг 8. Тестирование всех эндпоинтов

Проверим все маршруты с помощью curl. На Windows используйте двойные кавычки для JSON-данных (см. раздел 6.6)

Создание пользователя:

```
curl -X POST http://localhost:3000/api/users \
  -H 'Content-Type: application/json' \
  -d '{"name": "Алексей", "email": "alex@example.com"}'
```

Получение списка пользователей:

```
curl http://localhost:3000/api/users
```

Получение пользователя по ID (подставьте реальный _id):

```
curl http://localhost:3000/api/users/<_id>
```

Обновление пользователя:

```
curl -X PATCH http://localhost:3000/api/users/<_id> \
  -H 'Content-Type: application/json' \
  -d '{"name": "Новое имя"}'
```

Удаление пользователя:

```
curl -X DELETE http://localhost:3000/api/users/<_id>
```

Создание поста (укажите _id существующего пользователя):

```
curl -X POST http://localhost:3000/api/posts \
  -H 'Content-Type: application/json' \
  -d '{"title": "Мой пост", "content": "Текст", "author": "<user_id>"}'
```

Получение постов (author будет объектом, а не ObjectId):

```
curl http://localhost:3000/api/posts
```

Несуществующий маршрут (ожидаем 404):

```
curl http://localhost:3000/nonexistent
```

Ошибка валидации (ожидаем 400):

```
curl -X POST http://localhost:3000/api/users \
  -H 'Content-Type: application/json' \
  -d '{"name": "A"}'
```

Каждый запрос логируется в консоли сервера с датой, методом и URL. Ошибки валидации и дубликаты возвращают понятные HTTP-статусы (400, 404, 409). Promise.all() в маршруте GET /api/users выполняет запрос данных и подсчёт документов параллельно, что быстрее последовательного выполнения.

6.11 Практика

Все задания ниже расширяют приложение, которое вы создали в разделе 6.10. Выполняйте их последовательно — каждое следующее задание строится на предыдущем.

Задание 6.11.1: Добавление модели Comment (комментарии к постам)

Цель: добавить модель комментариев и связать её с постами и пользователями.

Что нужно сделать:

1. Создайте файл models/Comment.js:

```
import { Schema, model } from 'mongoose';


const commentSchema = new Schema({
    text: { type: String, required: true, minlength: 1, maxlength: 500
},
    user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
    post: { type: Schema.Types.ObjectId, ref: 'Post', required: true },
}, { timestamps: true });


export default model('Comment', commentSchema);
```

2. Создайте файл routes/comments.js с вложенными маршрутами:

- POST /api/posts/:postId/comments — добавить комментарий к посту.

- GET /api/posts/:postId/comments — получить все комментарии к посту с populate пользователя (name, email).

- DELETE /api/posts/:postId/comments/:commentId — удалить комментарий.

3. Подключите маршрут в server.js:

```
import commentsRouter from './routes/comments.js';
app.use('/api/posts', commentsRouter);
```

4. Измените маршрут DELETE /api/posts/:id — при удалении поста удаляйте все его комментарии:

```
await Comment.deleteMany({ post: req.params.id });
```

Подсказка для routes/comments.js:

```
import { Router } from 'express';
import Comment from '../models/Comment.js';

const router = Router();

// POST /api/posts/:postId/comments
router.post('/:postId/comments', async (req, res, next) => {
    try {
        const comment = await Comment.create({
            ...req.body,
            post: req.params.postId,
        });
        res.status(201).json(comment);
    } catch (err) { next(err); }
});
```

Проверьте:

```
curl -X POST http://localhost:3000/api/posts/<postId>/comments \
  -H 'Content-Type: application/json' \
  -d '{"text": "Отличный пост!", "user": "<userId>"}'

curl http://localhost:3000/api/posts/<postId>/comments
```

Задание 6.11.2: Фильтрация постов по тегам

Цель: добавить поле tags в модель Post и реализовать фильтрацию.

Что нужно сделать:

1. Добавьте в models/Post.js поле tags:

```
    tags: [{ type: String, lowercase: true, trim: true }],
```

2. Измените маршрут GET /api/posts — добавьте фильтрацию по тегу:

- ?tag=javascript — посты с указанным тегом.

- MongoDB автоматически ищет значение в массиве: Post.find({ tags: tag }).

3. Добавьте маршрут GET /api/posts/tags — получить список всех уникальных тегов:

Подсказка для списка уникальных тегов:

```
router.get('/tags', async (req, res, next) => {
    try {
        const tags = await Post.distinct('tags');
        res.json(tags);
    } catch (err) { next(err); }
});
```

Важно: маршрут /tags должен быть объявлен ДО маршрута /:id.

Подсказка для фильтрации по тегу:

```
const { tag, author } = req.query;
const filter = {};
if (tag) filter.tags = tag;
if (author) filter.author = author;
const posts = await Post.find(filter).populate('author', 'name email');
```

Проверьте:

```
curl -X POST http://localhost:3000/api/posts \
  -H 'Content-Type: application/json' \
  -d '{"title": "Основы JS", "content": "...", "author": "<userId>",
"tags": ["javascript", "tutorial"]}'

curl "http://localhost:3000/api/posts?tag=javascript"
curl http://localhost:3000/api/posts/tags
```

Задание 6.11.3: Статистика пользователя

Цель: добавить маршрут, возвращающий статистику по пользователю.

Что нужно сделать:

1. Добавьте в routes/users.js маршрут GET /:id/stats (перед маршрутом /:id).

2. Маршрут должен возвращать:

- totalPosts — общее количество постов пользователя.

- totalComments — количество комментариев пользователя (если выполнили задание 6.1).

- lastPost — заголовок и дата последнего поста.

- Если пользователь не найден — 404.

Подсказка:

```
const userId = req.params.id;
const user = await User.findById(userId);
if (!user) return res.status(404).json({ error: 'Пользователь не
найден' });

const [totalPosts, totalComments, lastPost] = await Promise.all([
    Post.countDocuments({ author: userId }),
    Comment.countDocuments({ user: userId }),
    Post.findOne({ author: userId })
        .sort({ createdAt: -1 })
        .select('title createdAt'),
]);
```

Ожидаемый ответ:

```
{
    "user": "Иван Иванов",
    "totalPosts": 5,
    "totalComments": 12,
    "lastPost": {
        "title": "Основы Node.js",
        "createdAt": "2024-01-15T10:30:00.000Z"
    }
}
```

Проверьте:

```
curl http://localhost:3000/api/users/<userId>/stats
```

Задание 6.11.4: Поиск постов

Цель: добавить маршрут полнотекстового поиска по постам.

Что нужно сделать:

1. Добавьте в routes/posts.js маршрут GET /search (перед маршрутом /:id):

- Параметр ?q=запрос — поиск по title и content.

- Если параметр q не указан — вернуть 400.

- Результаты с populate автора (name, email).

2. Используйте оператор $or и регулярные выражения для регистронезависимого поиска.

Подсказка:

```
router.get('/search', async (req, res, next) => {
    try {
        const { q } = req.query;
        if (!q) return res.status(400).json({ error: 'Параметр q
обязателен' });

        const posts = await Post.find({
            $or: [
                { title: { $regex: q, $options: 'i' } },
                { content: { $regex: q, $options: 'i' } },
            ],
        }).populate('author', 'name email');

        res.json(posts);
    } catch (err) { next(err); }
});
```

Проверьте:

```
curl "http://localhost:3000/api/posts/search?q=node"
curl "http://localhost:3000/api/posts/search?q=основы"
```

Вернёт все посты, в title или content которых есть указанное слово (регистронезависимый поиск).

Задание 6.11.5: Пагинация и сортировка

Цель: добавить параметры пагинации и сортировки во все маршруты получения списков.

Что нужно сделать:

1. Измените GET /api/users — добавьте поддержку параметров:

- ?page=1&limit=10 — пагинация (по умолчанию page=1, limit=10).

- ?role=admin — фильтрация по роли.

- ?sort=name или ?sort=createdAt — сортировка (по умолчанию createdAt).

- ?order=asc или ?order=desc — порядок (по умолчанию desc).

- В ответе возвращайте объект с users, total, page, pages.

2. Измените GET /api/posts — аналогично:

- Пагинация (?page, ?limit).

- Фильтрация по автору (?author=<userId>).

- Сортировка (?sort, ?order).

Подсказка:

```
const { page = 1, limit = 10, sort = 'createdAt', order = 'desc', role
} = req.query;
const filter = {};
if (role) filter.role = role;


const [users, total] = await Promise.all([
    User.find(filter)
        .sort({ [sort]: order === 'asc' ? 1 : -1 })
        .skip((page - 1) * limit)
        .limit(Number(limit)),
    User.countDocuments(filter),
]);

res.json({ users, total, page: Number(page), pages: Math.ceil(total /
limit) });
```

Проверьте:

```
curl "http://localhost:3000/api/users?page=1&limit=5"
curl "http://localhost:3000/api/users?role=admin"
curl "http://localhost:3000/api/users?sort=name&order=asc"
curl
"http://localhost:3000/api/posts?page=1&limit=5&sort=title&order=asc"
```

Задание 6.11.6 (дополнительное): Soft Delete

Цель: вместо физического удаления постов — помечать их как удалённые (soft delete). Это распространённый паттерн в реальных приложениях.

Что нужно сделать:

1. Добавьте в models/Post.js поле:

```
    deletedAt: { type: Date, default: null },
```

2. Измените маршрут DELETE /api/posts/:id — вместо findByIdAndDelete() используйте findByIdAndUpdate() и устанавливайте deletedAt: new Date().

3. Измените все маршруты GET для постов — добавьте условие { deletedAt: null } в фильтр, чтобы удалённые посты не попадали в выборку.

4. Добавьте маршрут PATCH /api/posts/:id/restore — для восстановления удалённого поста (установите deletedAt: null).

5. Добавьте маршрут GET /api/posts/deleted — для просмотра удалённых постов ({ deletedAt: { $ne: null } }).

Подсказка:

```
// Soft delete
router.delete('/:id', async (req, res, next) => {
    try {
        const post = await Post.findByIdAndUpdate(
            req.params.id,
            { deletedAt: new Date() },
            { new: true },
        );
        if (!post) return res.status(404).json({ error: 'Пост не
найден' });
        res.json({ message: 'Пост помечен как удалённый' });
    } catch (err) { next(err); }
});

// Восстановление
router.patch('/:id/restore', async (req, res, next) => {
    try {
        const post = await Post.findByIdAndUpdate(
            req.params.id,
            { deletedAt: null },
            { new: true },
        );
        if (!post) return res.status(404).json({ error: 'Пост не
найден' });
        res.json(post);
    } catch (err) { next(err); }
});
```

Важно:

Маршруты /deleted и /:id/restore должны быть объявлены ДО маршрута /:id.

Проверьте:

```
curl -X DELETE http://localhost:3000/api/posts/<postId> # soft delete
curl http://localhost:3000/api/posts                    # пост не виден
curl http://localhost:3000/api/posts/deleted            # пост в списке
curl -X PATCH http://localhost:3000/api/posts/<postId>/restore #
восстановление
```

Заключение

В этом модуле мы изучили:

- Что такое MongoDB, чем она отличается от SQL-баз данных, и три способа установки: Atlas, локально (Windows/macOS/Linux) и Docker.
- Mongoose — ODM-библиотеку, которая связывает Node.js-приложение с MongoDB и предоставляет удобный API для работы с данными.
- Схемы и модели: как описать структуру документа, задать валидаторы (required, minlength, enum, unique) и автоматические поля (timestamps).
- CRUD-операции: create(), find(), findById(), findByIdAndUpdate(), findByIdAndDelete() — и их использование в Express-маршрутах.
- Связи между моделями через ObjectId + ref и подгрузку данных через populate().
- Переменные окружения: .env, dotenv, --env-file, .env.example.
- Полноценное Express + Mongoose приложение: структура проекта, подключение к БД, роутеры, middleware, обработка ошибок.

Эти навыки — основа для построения любого серверного приложения на Node.js с базой данных. В следующих модулях мы добавим аутентификацию (JWT), загрузку файлов и развёртывание приложения.